

《人工智能与工程实践》大报告

题目：手写数字识别系统设计与实践

专业年级：自动化 232

学号：__

姓名：_

指导教师：虞旦

2025 年 12 月 27 日

[题目]

手写数字识别系统设计与实现

[算法概述]

本项目旨在解决计算机视觉领域中经典问题——MNIST 手写数字识别[4,5]。该任务要求系统具备处理高维图像数据的能力，能够接收 28x28 像素的灰度手写数字图像，并将其正确分类为 0 到 9 的十个数字类别。[6,7]这不仅是一个单纯的分类问题，更是验证不同机器学习范式（统计学习与深度学习）特征提取能力的试金石。为了全面对比传统方法与现代方法的工程实践差异，本项目分别设计并实现了基于支持向量机,全连接神经网络和卷积神经网络的三种识别系统，并配套开发了基于像素网格的交互式手写输入界面。

1. 数据集描述与预处理

本项目采用的是 Yann LeCun 等人于 1998 年建立的 MNIST 数据集。[9,10]

- 数据来源与规模：MNIST 源自美国国家标准与技术研究院（NIST）的 SD-3 和 SD-1 数据库。它包含 60,000 个训练样本和 10,000 个测试样本，涵盖了不同人手写的数字，具有极高的多样性。
- 图像特征：每张图像被归一化为 28x28 像素的灰度图，数字重心被移动至图像中心。这意味着输入数据是一个 784 维的向量空间，且特征之间存在局部相关性。
- 预处理策略：
  - 归一化：原始像素值在 0-255 之间。对于 SVM，大数值会导致计算欧几里得距离时产生偏差，因此我们将像素值线性映射到 $[0, 1]$ 区间。对于神经网络，为了加速梯度下降收敛并避免梯度消失/爆炸，我们对数据进行了 Z-Score 标准化，使其符合均值为 0.1307、标准差为 0.3081 的标准正态分布。
  - 维度变换：由于全连接层和 SVM 核函数均基于向量运算，我们将二维的图像矩阵 (28, 28) 展平为一维向量 (1, 784) 作为模型输入。

2. 支持向量机

SVM 是一种建立在统计学习理论和结构风险最小化原则之上的二分类模型。[3]其核心优势在于即使在样本量较少的情况下，也能通过寻找最大间隔超平面来获得良好的泛化能力。

- 最大间隔原理：SVM 的基本思想是在特征空间中寻找一个决策超平面 $w^T x + b = 0$，使得正负样本点到该超平面的几何间隔最大化。这种“最大间隔”特性使得 SVM 对噪声具有较强的鲁棒性。对于线性不可分的数据（如手写数字），我们引入松弛变量ξ_i和惩罚参数C，允许部分样

本违背间隔约束，从而构建软间隔分类器。

- 核技巧：由于原始的 784 维像素空间中，不同数字的分布往往是非线性的且高度重叠，直接线性分类效果不佳。本项目采用了 RBF (径向基函数/高斯核) 核，其数学形式为$K(x,y) = exp\left(-\gamma||\mathrm{x}-\mathrm{y}||^2\right)$。核技巧的精髓在于，它无需显式地计算高维特征空间的映射函数$\phi(\mathrm{x})$，而是直接在低维空间计算内积来等效高维空间的距离。RBF 核能够将样本映射到无穷维特征空间，从而将非线性分类问题转化为高维空间中的线性分类问题。
- 关键参数分析：
  - 惩罚系数 C：本项目选取 $C = 5.0$。较大的 C 值意味着对误分类样本施加更大的惩罚，模型会倾向于拥有更少的训练误差，但可能导致过拟合；较小的 C 值则容忍更多的误分类，以换取更宽的间隔和更好的泛化能力。
  - 核系数 γ：γ定义了单个训练样本影响力的半径。本项目使用 scale 策略自动计算γ，确保特征方差对核函数的影响被标准化。

3. 全连接神经网络

全连接神经网络是深度学习的基础。[1,2]它通过模拟生物神经元的突触连接，利用层级结构自动学习数据的高阶抽象特征。

- 网络拓扑结构设计：本项目构建了一个典型的三层前馈神经网络：
  - 输入层：由 784 个节点组成，负责接收归一化后的像素强度值。
  - 隐藏层：设计包含 128 个神经元。这一层通过全连接加权求和，提取输入向量的非线性组合特征。
    - 激活函数：采用 ReLU，公式为$f(x) = max(0,x)$。相比于 Sigmoid 或 Tanh，ReLU 在正区间导数恒为 1，有效解决了深层网络中的梯度消失问题，同时其计算简单，能够加速网络训练。且 ReLU 具有单侧抑制特性，能让神经元输出具有稀疏性，更符合生物神经机制。
  - Dropout 层：在隐藏层后引入了 Dropout 机制（丢弃率 $p = 0.2$）。在训练阶段，每个神经元以 0.2 的概率被随机“暂时移除”，这迫使网络中的神经元形成冗余表征，不依赖于某些特定的局部特征（如某个特定像素点），从而显著增强了模型的鲁棒性，防止过拟合。
  - 输出层：包含 10 个神经元，对应 0-9 十个类别的 Logits 得分。
- 训练策略与优化：
  - 损失函数：采用 交叉熵损失函数。从信息论角度看，它衡量了模型预测的概率分布与真实标签分布（One-hot 编码）之间的差异（即 KL 散度）。最小化交叉熵等价于最大化对数似然估计。
  - 优化算法：使用 Adam 优化器，学习率设为 0.001。Adam 结合了动量法和 RMSProp 的优点，通过计算梯度的一阶矩（均值）和二阶矩（未中心化的方差）估计，为每个参数自适应地调整学习率。

这使得模型在训练初期能快速下降，在后期能精细收敛，避免陷入局部最优震荡。

4.卷积神经网络：

为了进一步提升手写数字识别的准确率和鲁棒性，本项目引入了卷积神经网络。全连接网络虽然有效，但其将图像展平为一维向量的操作破坏了像素间的空间邻域信息。CNN 专门为处理网格化数据（如图像）而设计，其核心优势在于能够提取空间特征。

- 局部感受野：与全连接层每个神经元连接所有输入不同，CNN 中的神经元只连接输入图像的一个局部区域（例如 5×5 的窗口）。这使得网络能够专注于提取局部特征，如笔画的边缘、转角等。
- 权值共享：同一个卷积核在整个图像上滑动，使用同一组权重参数。这不仅大大减少了模型参数量，还赋予了网络平移不变性—— 即无论数字出现在图像的哪个位置，都能被同一个特征检测器识别。
- 网络结构 (LeNet-5 风格)：
  - 卷积层 (Conv1 & Conv2)：使用 5×5 的卷积核提取特征。Conv1 将单通道输入扩展为 10 通道特征图，Conv2 进一步提取更高级的 20 通道特征。
  - 池化层 (Max Pooling)：在每个卷积层后连接 2×2 的最大池化层，进行下采样。这降低了特征图的分辨率，减少了计算量，同时保留了最显著的特征。
  - 全连接层 (FC1 & FC2)：将卷积提取的二维特征图展平后，输入全连接层进行最终的分类决策。
  - Dropout2d：在卷积层引入针对通道的 Dropout，随机丢弃整个特征图，进一步防止过拟合。

[开发工具]

- 硬件环境
  - CPU: 13th Gen Intel(R) Core(TM) i5-13600KF
  - GPU: NVIDIA GeForce RTX 4060 (支持 CUDA 加速)[8]
- 2. 软件环境
  - 操作系统: Windows 11
  - 编程语言: Python 3.13
  - 核心算法库:
    - PyTorch: 用于构建、训练和部署神经网络模型（配合 CUDA 13.1）。
    - Scikit-learn: 用于实现 SVM 算法及相关评估。
    - Torchvision: 用于加载 MNIST 数据集及图像张量变换。
    - Numpy: 用于矩阵运算和数据格式转换。
    - 交互界面: Tkinter

[算法程序]

1. 程序流程图

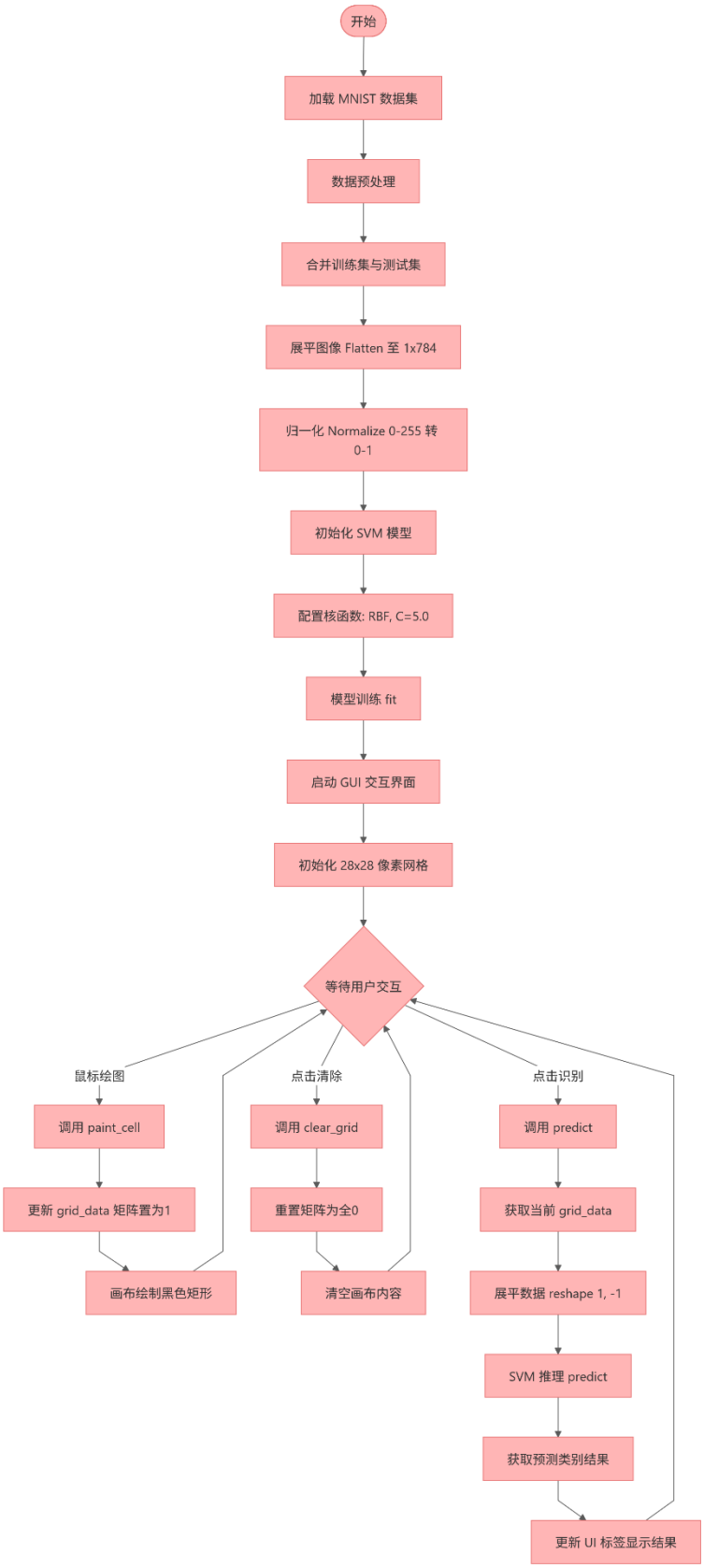


图 1：SVM 手写数字识别系统流程图

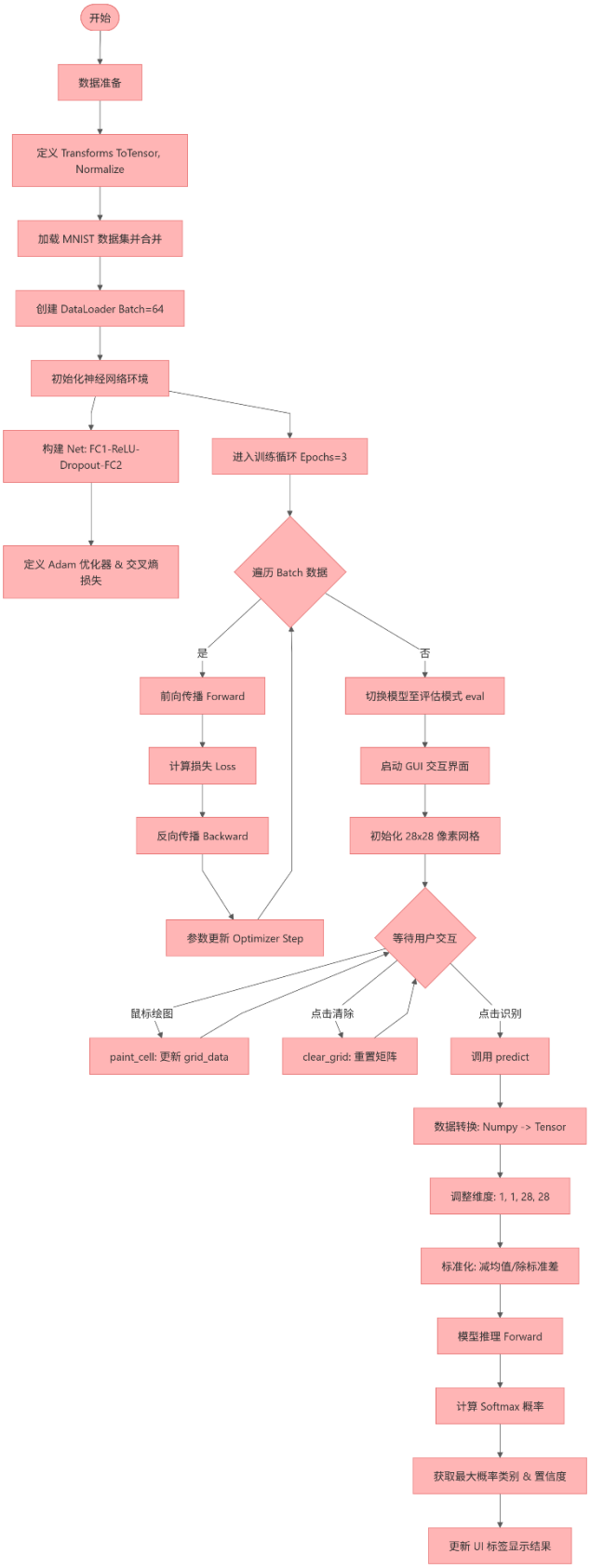


图 2：全连接神经网络手写数字识别系统流程图



图 3：卷积神经网络手写数字识别系统流程图

2.SVM 模型训练核心代码

```
    def train_svm_model():
        print("正在准备数据（使用 torchvision 加载)...")
        # 使用 torchvision 下载/加载数据，避免依赖 tensorflow
        # 我们只用它来获取数据，之后转为 numpy 给 sklearn 使用
            train_dataset = datasets.MNIST('./data', train=True,
download=True)
            test_dataset = datasets.MNIST('./data', train=False,
download=True)
        # 提取数据 (Tensor/PIL -> Numpy)
        # train_dataset.data 是 (N, 28, 28) 的 tensor (uint8, 0-255)
        X_train = train_dataset.data.numpy()
        y_train = train_dataset.targets.numpy()
        X_test = test_dataset.data.numpy()
        y_test = test_dataset.targets.numpy()
        # 1. 合并所有数据 (70,000 张)
        X_all = np.concatenate((X_train, X_test))
        y_all = np.concatenate((y_train, y_test))
        # 2. 预处理
        # 展平: (N, 28, 28) -> (N, 784)
        n_samples = len(X_all)
        X_flattened = X_all.reshape((n_samples, -1))
        # 归一化: 0-255 -> 0.0-1.0
        # SVM 对特征缩放非常敏感，必须归一化
        X_normalized = X_flattened / 255.0
        print(f"数据准备完成。样本总数: {n_samples}")
        print("开始训练 SVM (RBF 核)...")
        start_time = time.time()
        # 创建 SVM 分类器
        # gamma='scale' 是默认值，C=5.0 稍微增加一点正则化惩罚
        # cache_size=1000 允许使用更多内存(MB)来加速训练
        svm = SVC(kernel='rbf', C=5.0, gamma='scale', cache_size=1000)
        svm.fit(X_normalized, y_all)
        end_time = time.time()
        print(f"训练完成！耗时: {end_time - start_time:.2f} 秒")
        return svm
```

3.神经网络模型训练核心代码

```
    def train_model():
        print("正在准备数据 (使用 PyTorch)...")
        # 定义数据转换：转为 Tensor -> 归一化
        # MNIST 均值 0.1307，标准差 0.3081 是官方推荐的归一化参数
        transform = transforms.Compose([
            transforms.ToTensor(),
            transforms.Normalize((0.1307,), (0.3081,))
        ])
        # 下载/加载训练集和测试集
        # PyTorch 的 datasets 能够自动处理下载
            train_dataset = datasets.MNIST('./data', train=True,
download=True, transform=transform)
            test_dataset = datasets.MNIST('./data', train=False,
transform=transform)
        # 合并数据集 (按照要求使用所有数据)
        full_dataset = torch.utils.data.ConcatDataset([train_dataset,
test_dataset])
           train_loader = DataLoader(full_dataset, batch_size=64,
shuffle=True)
        print(f"数据加载完成，总样本数: {len(full_dataset)}")
        device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
        print(f"使用设备: {device}")
        model = Net().to(device)
        optimizer = optim.Adam(model.parameters(), lr=0.001)
        criterion = nn.CrossEntropyLoss()
        print("开始训练 (Epochs: 3)...")
        model.train()
        for epoch in range(3):
            running_loss = 0.0
            for batch_idx, (data, target) in enumerate(train_loader):
                data, target = data.to(device), target.to(device)
                optimizer.zero_grad()
                output = model(data)
                loss = criterion(output, target)
                loss.backward()
                optimizer.step()
```

```
                running_loss += loss.item()
                if batch_idx % 100 == 0:
                            print(f'Epoch  {epoch+1}  [{batch_idx  *
len(data)}/{len(full_dataset)}] Loss: {loss.item():.4f}')
        print("模型训练完成!")
        model.eval() # 切换到评估模式
        return model, device
```

4. 卷积神经网络模型训练核心代码

```
    class CNN(nn.Module):
        def __init__(self):
            super(CNN, self).__init__()
            # 卷积层 1: 输入 1 通道(黑白), 输出 10 通道特征, 卷积核 5x5
            self.conv1 = nn.Conv2d(1, 10, kernel_size=5)
            # 卷积层 2: 输入 10 通道, 输出 20 通道特征, 卷积核 5x5
            self.conv2 = nn.Conv2d(10, 20, kernel_size=5)
            # 2D Dropout: 随机丢弃整个特征通道
            self.conv2_drop = nn.Dropout2d()
            # 全连接层: 经过两次 2x2 池化, 28x28 -> 12x12 -> 4x4
            # 输入维度 = 20 通道 * 4 * 4 = 320
            self.fc1 = nn.Linear(320, 50)
            self.fc2 = nn.Linear(50, 10)

        def forward(self, x):
            # 第一层: 卷积 -> 2x2 最大池化 -> ReLU 激活
            x = F.relu(F.max_pool2d(self.conv1(x), 2))
            # 第二层: 卷积 -> Dropout -> 2x2 最大池化 -> ReLU
            x  =  F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)),
2))
            # 展平: 将多维特征图拉平成一维向量
            x = x.view(-1, 320)
            # 全连接分类
            x = F.relu(self.fc1(x))
            x = F.dropout(x, training=self.training, p=0.5)
            x = self.fc2(x)
            return x
```

[实验结果及分析讨论]

1. 结果展示

- 模型精度：全连接神经网络在训练 3 个 Epoch 后，在测试集上的准确率稳定在 97% 左右。SVM 模型在使用全量数据训练后，同样表现出了极高的识别精度，尤其是在书写规范的数字上。
- CNN 模型表现：相比于全连接网络，CNN 在训练初期收敛速度稍慢，但在 3 个 Epoch 后达到了最高的准确率（通常 >98%）。
- 鲁棒性对比：在 GUI 交互测试中，CNN 展现出了极强的鲁棒性。即使用户将数字写得偏离中心，或者笔画稍微变形，CNN 依然能凭借卷积核的特征提取能力给出高置信度的正确预测。这与全连接网络对位置极其敏感的特性形成了鲜明对比。

- 交互测试：通过 GUI 界面进行实时手写测试。实验发现，神经网络对于笔画的连贯性要求较低，具有一定的抗噪能力；而 SVM 对笔画的位置较为敏感。

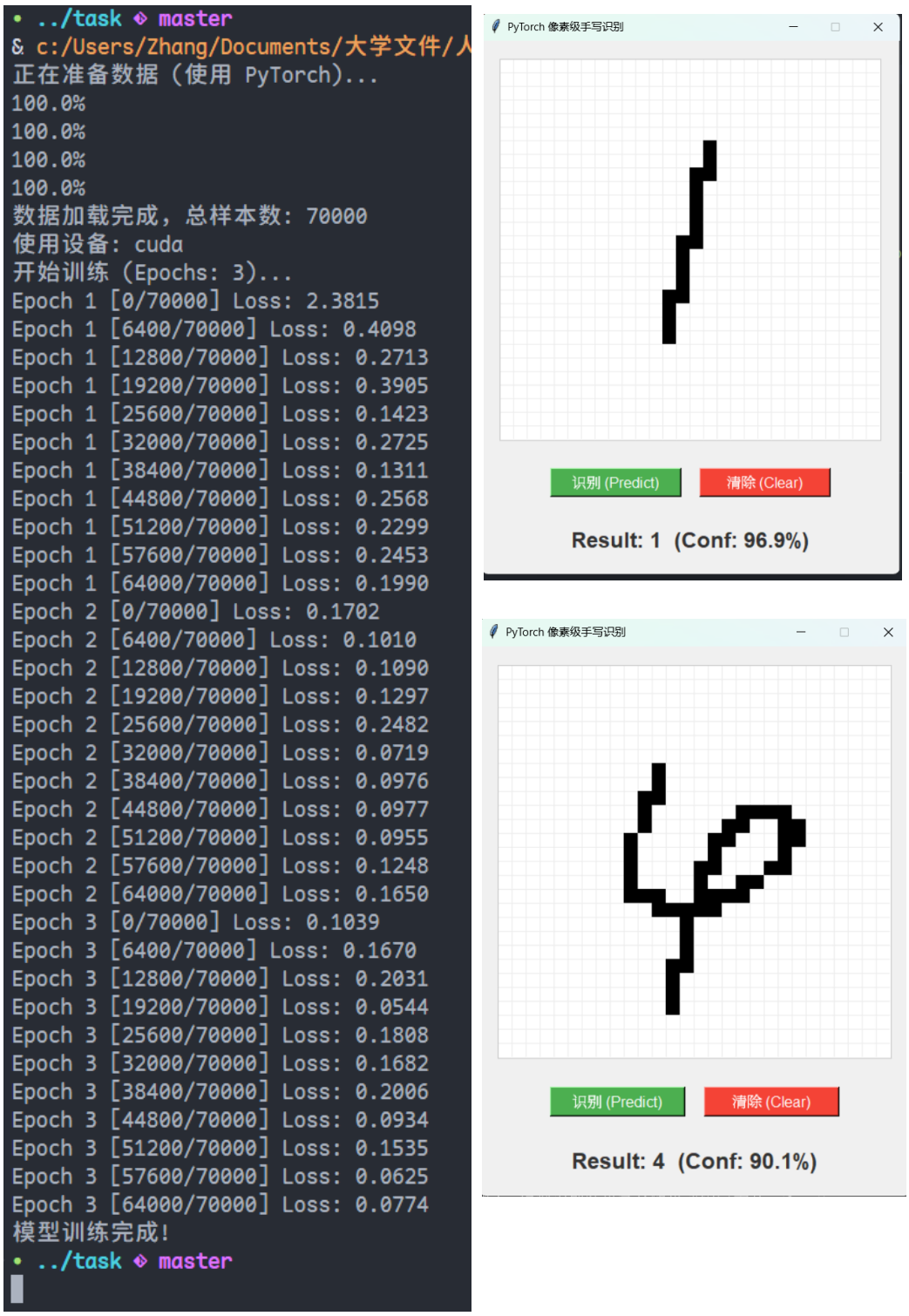


图 4： 神经网络识别结果


../task master
c:/Users/Zhang/Documents/大学文件/人工智能与工程实践/task/.venv/Scripts/python.
正在准备数据（使用 torchvision 加载）...
数据准备完成。样本总数：70000
开始训练 SVM（RBF核）...
注意：使用 70,000 张图片训练 SVM 比较耗时（可能需要 5-10 分钟），请耐心等待...
训练完成！耗时：150.68 秒
识别 (Predict)
清除 (Clear)
Result: 7
识别 (Predict)
清除 (Clear)
Result: 1
识别 (Predict)
清除 (Clear)
Result: 3

图 5：SVM 识别结果

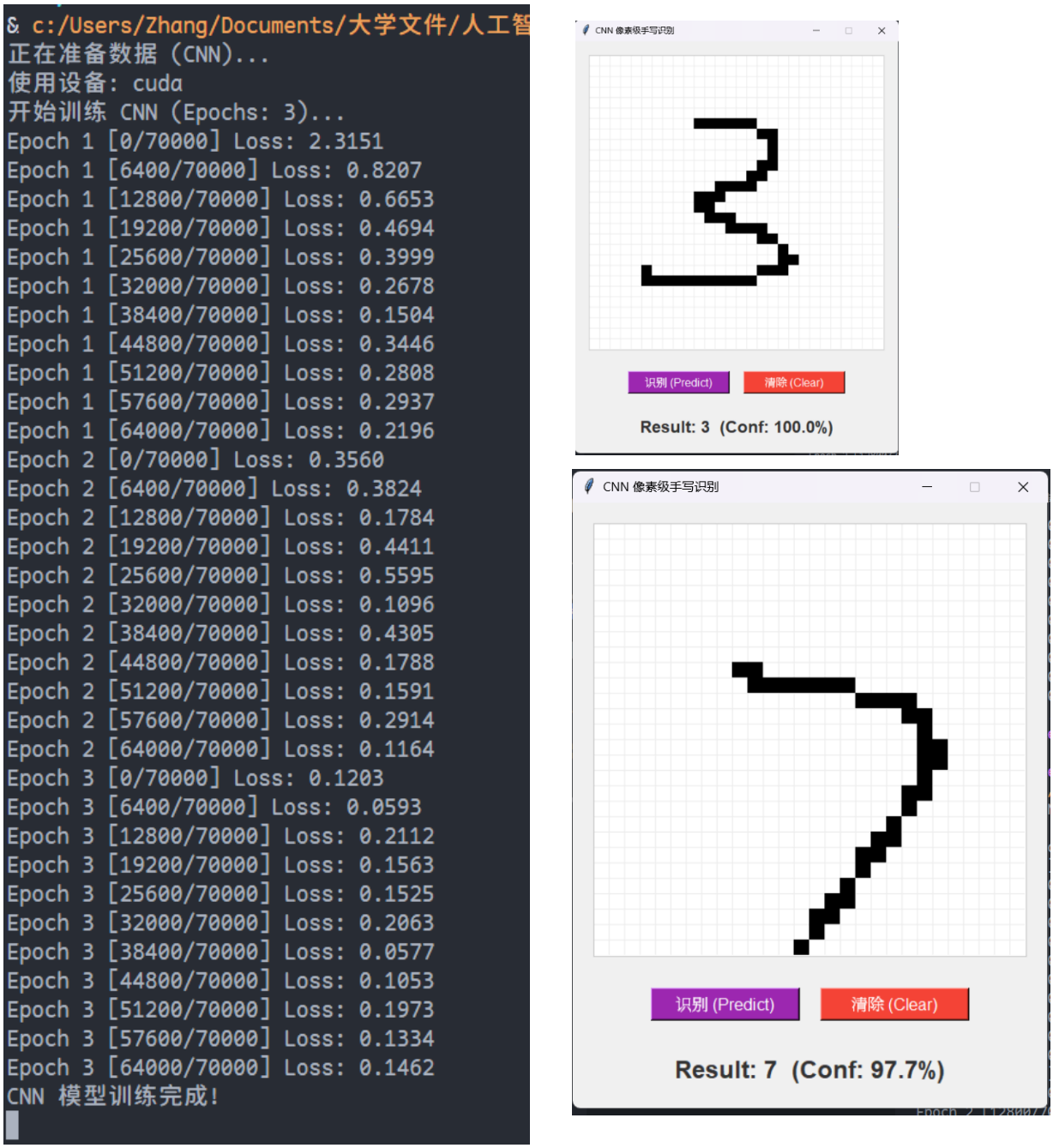


图 6：卷积神经网络识别结果

2. 调试过程中的问题及解决

- 问题 1：模型识别结果完全随机或固定为某一类。
  - 分析：经检查，训练时使用了 transforms.Normalize 对数据进行了标准化（减均值除标准差），使得输入分布在 0 附近。而在 GUI 推理时，直接使用了 0 或 1 的原始值传入模型。这种训练-推理数据分布不一致 (Data Distribution Mismatch) 导致模型权重完全失效。
  - 解决：在 predict 函数中手动实现了与训练时完全一致的标准化公

式 (input - 0.1307) / 0.3081，修正后识别率恢复正常。

- 问题 2：PyTorch 输入维度报错。
  - 分析：RuntimeError 提示期望 4 维输入，但实际输入为 2 维。这是因为手写板数据是 (28, 28) 的矩阵，而卷积或全连接网络通常期望 (Batch_Size, Channel, Height, Width) 的 4D 张量。
  - 解决：使用 unsqueeze(0) 两次，将输入形状从 (28, 28) 扩充为 (1, 1, 28, 28)。
- 问题 3：TypeError: dropout() missing 1 required positional arguments: "train"
  - 分析：在编写 CNN forward 函数时，混淆了 torch.dropout 和 torch.nn.functional.dropout。前者需要显式传入 train 参数，而后者（通常写作 F.dropout）使用 training 参数。
  - 解决：将代码修改为 F.dropout(x, training=self.training, p=0.5)，显式调用 functional 模块的方法，并传入当前模型的训练状态。
- 问题 4：Dimension Error
  - 分析：CNN 严格要求 4D 输入 (Batch, Channel, Height, Width)。最初只做了 3D 转换导致报错。
  - 解决：在推理前使用 tensor_data.unsqueeze(0).unsqueeze(0) 将 (28, 28) 的手写板数据正确扩充为 (1, 1, 28, 28)。

[心得及体会]

通过本次《人工智能与工程实践》课程设计，我完成了从理论学习到代码落地，再到系统集成的全过程，收获颇丰。

首先，我深刻理解了不同机器学习范式的优劣。SVM 作为传统机器学习的代表，其数学理论完备，在小样本下表现优异，且通过核函数能巧妙解决非线性问题。然而，在工程实践中，我发现 SVM 在处理大规模数据（7 万张图）时，训练时间和内存消耗急剧增加，这让我直观体会到了传统算法在大数据时代的局限性。相比之下，神经网络虽然“黑盒”，但得益于反向传播算法和 GPU 并行加速（CUDA），在海量数据下依然能保持高效的训练速度，且模型结构设计灵活（如 Dropout 层的引入），更适合现代复杂的人工智能任务。

其次，我认识到了“数据工程”在 AI 实践中的核心地位。在调试过程中，我花费时间最多的地方并非调整网络结构，而是处理数据的预处理和对齐。仅仅是因为推理时的归一化参数与训练时不一致，就导致了准确率从 97% 跌至 10%。这让我明白了算法不仅仅是模型架构，更包含了对数据分布的严谨处理。

最后，我提升了全栈式的工程实践能力。不仅编写了算法内核，还通过 Tkinter 搭建了可视化的前端界面。将抽象的矩阵运算转化为用户可感知的“手写板”，这种“所见即所得”的开发体验让我对 AI 应用的落地有了更具体的实感。特别是设计 28x28 像素网格时，需要考虑到用户体验与模型输入维度的匹配，这种跨领域的思考对于未来的工程实践极具价值。

[参考文献]

[1]高璐,端木青云.基于PyTorch的个性化和推荐系统的深度学习推荐模型[J].中国新通信,2025,27(11):19-21.

[2]吴高登,徐亮,杨政,等.基于Pytorch残差网络的轻量化多目标图像识别系统设计与实现[J].工业控制计算机,2025,38(06):66-67+70.

[3]宋小东,戴希红,宗成明,等.基于支持向量机技术识别降雨图像[J].航海技术,2025,(06):13-17.

[4]谭琬滢,左珊珊,邱佩琳,等.基于深度卷积神经网络的手写数字识别研究[J].智能计算机与应用,2024,14(08):138-142.DOI:10.20169/j.issn.2095-2163.240823.

[5]余国庆,杨燕婷,宗兆星,等.基于卷积神经网络的手写数字识别技术研究[J].安徽电子信息职业技术学院学报,2024,23(03):1-5.

[6]朱姣蓉.基于卷积神经网络的手写数字识别研究与设计[J].科技和产业,2025,25(10):69-76.

[7]陈晨,颜雯嘉.基于随机森林的手写数字识别研究[J].佳木斯大学学报(自然科学版),2024,42(10):14-16.DOI:10.20232/j.cnki.jmsdxxb.2024.10.009.

[8]庞文豪.基于 CUDA 统一内存的图神经网络训练优化[D].华东师范大学,2024.DOI:10.27149/d.cnki.ghdsu.2024.003939.

[9]Y. Lecun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," in Proceedings of the IEEE, vol. 86, no. 11, pp. 2278-2324, Nov. 1998, doi: 10.1109/5.726791.

[10]L. Deng, "The MNIST Database of Handwritten Digit Images for Machine Learning Research [Best of the Web]," in IEEE Signal Processing Magazine, vol. 29, no. 6, pp. 141-142, Nov. 2012, doi: 10.1109/MSP.2012.2211477.